Building a fraud detection system, file by file

A build-order tour of a sequence-model fraud detection system: what each file is for, why it exists separately, and the lines where the obvious version is wrong.

Contents

Part 0: what forces the design 4

Part 1: the contract layer 5

src/data/schema.py — build this first, it's 100 lines and everything imports it . . 5

src/config.py — every number that changes a reported result 7

Part 2: the world — src/simulate.py 10

The universe: power laws, because that's the point 10

Traffic: a thinned Poisson process 11

The part that got rebuilt: making fraud hard enough to be worth detecting . . . 11

Adversarial drift: each mechanism attacks its own defence 13

Label noise, applied last 13

src/data/loaders.py — the seam that keeps real data optional 14

Part 3: time — src/data/splits.py 15

Part 4: the hard part — src/data/features.py **16**

The single most important structural decision . . . 16

Timestamps: the one-line fix for a bug you'd never find . . . 17

The rewrite: bounded history . . . 18

The window trick . . . 19

The line that enforces causality . . . 20

Amount z-score: two defensive constants . . . 20

Entity novelty, vectorised . . . 21

Out-of-fold target encoding, and the leak the tests caught . . . 22

The serving path . . . 23

What I'd change . . . 24

Part 5: windows — src/data/sequences.py **24**

The lexsort trap . . . 24

Building the windows . . . 25

The gather . . . 26

Weighted sampling . . . 26

Part 6: the leakage tests — write these now, not at the end **27**

Part 7: the baseline, and how we'll judge everything **29**

src/evaluation.py — before the models, because we can't read results without it 29

src/baselines.py — the number to beat, built to win . . . 31

src/artifacts.py — the boundary between training and everything else . . . 33

Part 8: the models — src/models/ **34**

src/models/base.py — the contract every architecture signs . . . 34

src/models/tcn.py — causality by construction . . . 36

src/models/transformer.py — the mask, and the NaN nobody expects . . . 37

Part 9: training — src/train.py 38

Part 10: calibration — src/calibration.py 40

Part 11: cost — src/policy.py 41

The mistake 43

Part 12: monitoring — src/monitoring.py 44

The negative result, which is the interesting part 45

Part 13: interpretability — src/explain.py 46

Part 14: serving — src/api/ 47

src/api/service.py — the arithmetic that makes parity work 48

src/api/audit.py — the record that has to exist before the response 50

src/api/app.py — which failures crash, which don't 51

Part 15: orchestration — src/experiment.py and src/report.py 52

Part 16: packaging, CI, Docker 53

Part 17: what's messy, and what I'd do differently 54

Rebuild checklist 56

We're sitting in front of an empty folder. Over the next few hours we're going to build something that decides whether a card transaction is fraudulent, in under 50 milliseconds, and — more importantly — that can *prove* it isn't lying to us about how well it works.

That second part is the whole job. Fraud detection is unusually good at producing numbers that look excellent and mean nothing. The class is rare, so accuracy is meaningless. The data is temporal, so a random split leaks. The labels arrive weeks late, so anything using "was the last transaction fraud?" is untrainable in production. And the adversary adapts, so a stationary holdout measures the one regime guaranteed not to persist.

So the order we build things in is not arbitrary. We're going to build the *measurement apparatus* before we build anything worth measuring, because otherwise we won't be able to tell whether what we built works.

A note on how to read this: I'll explain why a piece has to exist before showing it. When I quote code I'll give you the real file and line. When a line encodes a decision rather than mechanics, I'll slow down and show you the tempting wrong version too — the near-misses are where the learning is, and in this domain the near-misses usually make your numbers *better*, which is why they survive review.

Part 0: what forces the design

Before we type anything, four facts about the problem dictate most of the structure.

Fact one: the positive class is ~0.6% of rows. That kills accuracy as a metric, makes log-loss a bad early-stopping signal, and means every metric we report needs a variance estimate, because a few hundred test positives is not a lot to average over.

Fact two: a transaction is not independently fraudulent. A £3 streaming payment is unremarkable. Nine of them in four minutes from an unfamiliar device is card testing. The signal lives in the relationship between transactions, which means our unit of prediction is "transaction plus its history", not "transaction".

Fact three: labels lag. A chargeback is confirmed weeks after the authorisation. At scoring time we genuinely do not know whether last Tuesday's purchase turned out to be fraud. Any feature built from prior labels is a feature we cannot compute in production.

Fact four: an authorisation request is bounded. Whatever features we invent, the live system has to compute them from a payload that arrives over HTTP. If we build features that need a customer's entire history, we've built something we cannot deploy — or worse, something we deploy *differently* from how we trained it, which fails silently.

Fact four is the one that bites hardest and it bit this project mid-build. We'll get there.

Part 1: the contract layer

`src/data/schema.py` — build this first, it's 100 lines and everything imports it

Why it exists. Every module downstream needs to agree on what a transaction *is*. Without a single definition, `simulate.py` writes `merchant`, `features.py` reads `merchant_id`, and you find out at runtime three hours later. More importantly, we want synthetic and real datasets to be interchangeable: a real dataset gets adapted *into* this schema by its loader, and nothing downstream knows or cares which it's looking at.

Why it's separate. It has zero dependencies. Everything else has to import it, so if it lived inside (say) the simulator, the serving code would import the simulator to score a transaction. That's how you end up shipping numpy's RNG to production.

The core of it is a class of string constants, and one distinction that carries real weight:

```
# src/data/schema.py:52-74 (abridged)
class TransactionSchema:
    """Column names for the canonical transaction table.

    A row is one authorisation request. Everything in
      ``REQUEST_FIELDS`` is known
    at scoring time; everything in ``LABEL_FIELDS`` arrives later (a
      chargeback
    is typically confirmed weeks after the authorisation) and must
      never be used
    as a feature, including for prior transactions in a sequence.
    """
    TXN_ID = "txn_id"
    CUSTOMER_ID = "customer_id"
    TIMESTAMP = "timestamp"
    ...
    IS_FRAUD = "is_fraud"
    FRAUD_TYPE = "fraud_type"
```

That split between `REQUEST_FIELDS` and `LABEL_FIELDS` isn't documentation. It's the contract that Fact three imposes, written down in the one place every module reads. When we get to the leakage tests we'll assert it mechanically.

Then three integers that quietly constrain every embedding table we'll ever build:

```
# src/data/schema.py:100-102
PAD_INDEX = 0
UNK_INDEX = 1
N_RESERVED_CODES = 2
```

Why two reserved indices and not zero. Padding needs an index because our sequences are variable-length and we're going to pad them into a rectangle. Unknown needs a *separate* index because "a merchant

I have never seen before" is genuinely informative — first-time merchants are riskier — and collapsing it into padding would teach the model that unfamiliar merchants and non-existent transactions are the same thing.

The tempting wrong version is to use `0` for both, or to map unknowns to `-1`. The `-1` version blows up the moment it hits `nn.Embedding`, which is at least loud. The shared-`0` version trains fine and quietly wastes a genuinely useful feature. Guess which one survives code review.

`FraudType` is a `StrEnum` — an enum whose members *are* strings, so `FraudType.CARD_TESTING == "card_testing"` is `True`. That means we can write it into a pandas column and compare against raw strings without conversion, while still getting an authoritative list of valid values.

`src/config.py` — every number that changes a reported result

Why it exists. Scatter `0.005` across six files and you have six things to change and five you'll forget. More subtly: when the write-up says "we use a 0.5% review capacity", the reader needs to be able to check that claim against one place.

Why it's separate rather than constants at the top of each module. Because the README quotes these values. A config module is the interface between the prose and the code, and it's the only way the two can be kept honest.

It's a set of frozen dataclasses. Two constants sit at module level because they're used everywhere:

```
# src/config.py:23-27
SEEDS: tuple[int, ...] = (0, 1, 2)

#: Review capacities a real fraud team can actually work: fraction of
  the
#: scored population an analyst queue can absorb per day.
REVIEW_CAPACITIES: tuple[float, ...] = (0.001, 0.005, 0.01)
```

`SEEDS` being a tuple of three is a claim: every number we report is a mean over three runs. It's the minimum defensible number and — spoiler — for one of our headline metrics it turns out not to be enough. We'll report that rather than hide it.

`REVIEW_CAPACITIES` encodes something about the world, not about the model. A fraud team has N analysts, each of whom can work M alerts a day. That fixes the alert budget as a *fraction of volume*, and the only precision that matters is precision at that fraction. Reporting precision at a 0.5 probability threshold — the default everyone reaches for — answers a question nobody in the building has.

Now the values that took actual thought.

```
# src/config.py:60-61
    drift_start_frac: float = 0.85
    unreported_fraud_rate: float = 0.04
```

`drift_start_frac = 0.85` puts the adversarial change in the last 15% of the timeline. Move it earlier and it lands inside the training window, so the model learns the drifted pattern and there's nothing to detect. Move it later and the test set has too few drifted transactions to measure. It needs to land after training ends and with enough test period left to be visible — with a 60/15/25 split, 0.85 puts it about two-thirds of the way through test.

`unreported_fraud_rate = 0.04` means 4% of fraudulent transactions carry a *legitimate* label, because the victim never disputed them. This isn't noise for its own sake. It's the reason real fraud models plateau short of perfect separation, and without it this synthetic problem is

solvable to PR-AUC 1.0. Set it to 0 and every model scores identically well and the comparison measures nothing.

```
# src/config.py:81-83
    train_frac: float = 0.60
    val_frac: float = 0.15
    embargo_days: float = 1.0
```

The embargo is the interesting one. A transaction one second after the training cutoff builds its rolling features from transactions the model trained on. One day of gap makes that structurally impossible for our longest window (7 days... which it doesn't fully cover — more on that when we hit the honest-limitations section).

```
# src/config.py:129-130
    focal_alpha: float = 0.25
    pos_sample_weight: float = 40.0
```

`focal_alpha = 0.25` is the RetinaNet default and it does something counter-intuitive that we'll measure later: it weights the *positive* class at 0.25 and the negative at 0.75. People assume alpha up-weights the rare class. It doesn't — it's there to counteract the `(1-p)^gamma` term, which already up-weights hard examples, most of which are positives. Get this backwards and your probabilities are systematically wrong in a direction that ranking metrics won't show you.

`pos_sample_weight = 40.0` takes a 0.6% positive rate to roughly 18% in the resampled stream. Not 50% — that's a much more aggressive distortion, and the model ends up estimating a base rate two orders of magnitude away from reality.

```
# src/config.py:153, 175
    false_positive_cost: float = 8.0
    latency_budget_p99_ms: float = 50.0
```

£8 for declining a good customer is a guess. It's *documented* as a guess, and we'll sweep it from £2 to £30 rather than defend the specific number. The 50 ms budget is derived: a card authorisation has a few

hundred milliseconds end to end before the acquirer times out, shared between network, decisioning and the issuer's core. The model is one hop inside that.

Part 2: the world — `src/simulate.py`

Why this comes before any model. Two reasons, one obvious and one not.

The obvious one: we need data.

The non-obvious one: **no public fraud dataset tells you which attack produced a given fraud.** IEEE-CIS has `isFraud`, a binary. That means you cannot ask the single most useful question about a fraud model — "which attack types is it blind to?" — because an adversary finds the blind spot first and aggregate metrics hide it completely. If we generate the data, we know the mechanism, and `src/evaluation.py` can break recall down by attack type.

Why it's one file and not a package. It's ~900 lines but it's one coherent thing with a single entry point, and splitting it would mean the merchant universe, the customer universe and the fraud injectors live apart while being tightly coupled through the RNG. One file, clearly sectioned.

The universe: power laws, because that's the point

```
# src/simulate.py:160-174
def _power_law_weights(n: int, exponent: float, rng:
  np.random.Generator) -> np.ndarray:
    ranks = np.arange(1, n + 1, dtype=np.float64)
    weights = ranks ** (-exponent)
    rng.shuffle(weights)
    return weights / weights.sum()
```

Three lines, and the middle one is the entire justification for using embeddings at all. `ranks ** (-exponent)` is Zipf: merchant #1 gets weight 1, #2 gets 2^-0.95, #100 gets 100^-0.95. A handful of merchants carry most volume and a long tail is seen a few times each.

If we'd sampled merchants uniformly, one-hot encoding would work fine, target encoding would be stable, and the "learned embeddings help with high-cardinality categoricals" claim would be untestable. The `rng.shuffle` matters too: without it, merchant ID would correlate with popularity, and the model could learn "low ID = popular" from the integer code alone.

Traffic: a thinned Poisson process

```
# src/simulate.py:330
    keep = rng.random(size=n_candidates) < accept
```

We draw twice as many candidate transaction times as we need, uniformly, then accept each with a probability that depends on hour-of-day and day-of-week. That gives realistic clustering — people shop in the evening, more at weekends — without simulating an actual point process.

The tempting wrong version is to draw times uniformly and be done. You get a model that can't use hour-of-day, because there's no diurnal signal to use, and then "3am transaction" isn't suspicious and one of our fraud mechanisms becomes undetectable for reasons that have nothing to do with the model.

The part that got rebuilt: making fraud hard enough to be worth detecting

Here's the thing I'd tell you before you write the fraud injectors, because I didn't know it when I wrote them the first time.

The first version of this simulator produced a problem where the LightGBM baseline scored PR-AUC 1.0000 on the pre-drift test period. Perfect. Which means every model scores perfect, and the entire comparison — the whole point of the project — measures nothing.

The bug wasn't in the fraud. It was in the *legitimate* traffic being too clean. Every fraudulent transaction used a device the customer had never used. Every legitimate transaction used the customer's one device. So "unseen device" was a perfect classifier, and no amount of clever modelling was going to be measured against that.

The fix is a list of confounders, each of which exists to blunt exactly one give-away:

- Customers hold 2–4 devices with lopsided usage, and ~5% of online transactions come from a device never seen before — a new browser, a borrowed laptop. Now "unseen device" is evidence, not proof.
- ~10% of customers make legitimate bursts of small purchases inside an hour. That's what card testing has to hide behind.
- ~2% of legitimate purchases are large one-offs; travel produces foreign countries and unfamiliar merchants.
- Attackers reuse the victim's own device 15–25% of the time (malware, session hijack).

And the device fingerprint:

```
# src/simulate.py:467
    if drifted or rng.random() > 0.35:
        return str(rng.choice(np.array(DEVICE_TYPES), p=_DEVICE_TYPE_P))
    return str(rng.choice(np.array(FRAUD_DEVICE_TYPES),
      p=_FRAUD_DEVICE_P))
```

Read that condition carefully. Pre-drift, only 35% of fraud sessions draw from the "automation fingerprint" pool (headless clients, emulators); the other 65% look like ordinary consumer devices. Post-drift, *none* do.

The tempting version is `if drifted: consumer else: fraud_pool` — a clean separation. That gives you a feature that's 100% predictive pre-drift and 0% post-drift, which is a step function, not a signal. 35% gives you something a model has to *combine* with other evidence, which is the situation we're actually trying to study.

Adversarial drift: each mechanism attacks its own defence

The drift isn't "the distribution shifts". It's targeted:

- Card testing raises amounts into the normal range and stretches the burst over hours instead of minutes — defeating velocity-plus-small-amount rules.
- Account takeover mimics the victim's own amount distribution and reuses a known device — defeating "new device plus big amount".
- Merchant compromise fans out across long-tail merchants with a longer delay — eroding any merchant-risk-rate feature.

That's what makes the monitoring section testable rather than decorative. Random noise injected into the test period would drift the data; it wouldn't attack the model.

Label noise, applied last

```
# src/simulate.py:820 (context)
        hidden = np.zeros(len(df), dtype=bool)
        hidden[rng.choice(fraud_rows, size=n_unreported,
          replace=False)] = True
        df.loc[hidden, S.IS_FRAUD] = np.int8(0)
        df.loc[hidden, S.FRAUD_TYPE] = FraudType.NONE.value
```

Note this runs *after* concatenation and sorting, on the final frame. It has to: we're flipping a random 4% of all fraud, and doing it per-mechanism during generation would change the relative mechanism mix.

Note also the `df.loc[boolean_array, col]` rather than `df.iloc[integer_array, col_position]`. We reset the index just

above this. The `.iloc` version with a positional column index is what I wrote first, and mypy rejected it — correctly, because `iloc` with a `(row_array, column_int)` tuple is not a supported signature and pandas' behaviour there is version-dependent.

`src/data/loaders.py` — the seam that keeps real data optional

Why it's separate from the simulator. Because "where does a DataFrame come from" and "how do we fabricate one" are different jobs, and everything downstream should depend only on the first. `load_dataset("synthetic")` and `load_dataset("ieee-cis")` return the same columns; nothing after this file knows which it got.

The interesting decision is what happens when the file isn't there:

```
# src/data/loaders.py (in _require)
    missing = [p for p in paths if not p.exists()]
    if not missing:
        return
    instructions = DOWNLOAD_INSTRUCTIONS[dataset].format(raw=paths[0].pa
      rent.resolve())
    listing = "\n".join(f"  - {p}" for p in missing)
    raise DatasetNotAvailableError(
        f"Missing files for dataset '{dataset}':\n{listing}\n{instructio
          ns}"
    )
```

No download, no credential prompt, no cache. The error message contains the URL, the filename and the absolute path to drop it in. A library that tries to authenticate to Kaggle on your behalf is a library that fails in CI, fails behind a proxy, and stores a token you didn't ask it to store.

The two real-data adapters both lose something, and both say so on the loader rather than in a footnote. IEEE-CIS has no cardholder key at all, so `customer_id` becomes a `card1/card2/addr1` composite — which over-merges (two cards sharing a BIN and postcode collide) and under-merges (a reissued card looks new). ULB is worse for our purposes: 28 anonymised PCA components and no entity identifiers

whatsoever, so per-customer sequences cannot be constructed. It maps every row to one pseudo-customer and logs a warning saying the sequence models degenerate to a global stream.

That last one is worth including precisely *because* it's the dataset everyone reaches for. Being explicit about why it cannot answer this project's question is more useful than quietly running on it and reporting a number.

Part 3: time — `src/data/splits.py`

Why it's its own file (it's 130 lines and does one thing): because the split is a *claim* the whole project rests on, and it needs to be somewhere a reviewer can find it in five seconds. Buried at the bottom of a training script, nobody checks it.

```
# src/data/splits.py:93-101
    train_end = t0 + span * cfg.train_frac
    val_start = train_end + embargo
    val_end = t0 + span * (cfg.train_frac + cfg.val_frac)
    test_start = val_end + embargo

    return TemporalSplit(
        train=(ts <= train_end).to_numpy(),
        val=((ts >= val_start) & (ts <= val_end)).to_numpy(),
        test=(ts >= test_start).to_numpy(),
```

The tempting wrong version, and it is *extremely* tempting because scikit-learn hands it to you:

```
train, test = train_test_split(df, test_size=0.25, random_state=42)  #
  WRONG
```

This is the single most common way fraud papers report numbers they can't reproduce. It puts a customer's Thursday in train and their Wednesday in test. Every rolling feature we're about to build then has partial knowledge of the test set through the training set's view of the

same customer. Your PR-AUC goes up. Everything looks better. Nothing works.

Note that the three masks don't cover every row — the embargo periods belong to no split. Those transactions still *exist* in the frame and still appear in later rows' history windows, which is correct: history is history. They're just never scored or trained on.

Also note `span * cfg.train_frac` splits by **time**, not by row count. With seasonal volume the two differ. Time is the right one: it's what a deployment boundary actually looks like.

Part 4: the hard part — `src/data/features.py`

This is the file where the project is won or lost, it's the longest one, and it's the one that got substantially rewritten mid-build. I'll take it slowly.

Why it's separate from `sequences.py`. Features answer "what do we know about this transaction at this instant". Windows answer "which transactions does the model see". Those change for different reasons — you'd add a feature without touching windowing, and change window length without touching features. And critically, the serving path needs the feature code but builds its own windows.

The single most important structural decision

```
# src/data/features.py:202-212 (signature)
def customer_causal_features(
    ts_sec: np.ndarray,
    amount: np.ndarray,
    merchant_key: np.ndarray,
    ...
    lookback: int = DEFAULT_LOOKBACK,
) -> dict[str, np.ndarray]:
```

This function takes numpy arrays, not a DataFrame. That signature constrains everything downstream, and it's deliberate.

The batch pipeline has a DataFrame and will hand us columns. The serving path has a JSON payload of ~60 records and would pay 1–2 ms of pure pandas overhead to build a DataFrame it immediately tears apart again. If this function took a DataFrame, serving would either eat that cost or — far more likely — someone would write a "fast path" that reimplements the features in numpy.

And that reimplementation is *the* classic production fraud bug. Two feature implementations that agree on the happy path and disagree on the boundary between "seen this merchant" and "seen this merchant within the window". The offline metrics stay excellent. The deployed model degrades. Nothing alerts.

One implementation, called by both paths, verified equal by a test. That's the whole design and it starts with this signature.

Timestamps: the one-line fix for a bug you'd never find

```
# src/data/features.py:150-159
def to_epoch_seconds(nanoseconds: np.ndarray) -> np.ndarray:
    """Convert integer nanosecond timestamps to seconds at microsecond
      resolution."""
    return (np.asarray(nanoseconds, dtype=np.int64) //
      _NS_PER_US).astype(np.float64) / 1e6
```

pandas timestamps carry nanoseconds. JSON datetimes carry microseconds at best. If the batch path does `df.timestamp.astype("int64") / 1e9` and the serving path does `pd.Timestamp(record["timestamp"]).value / 1e9`, the two disagree in the last few decimal places of every time delta.

Does it matter numerically? Not remotely — we're talking about nanoseconds inside `log1p(seconds)`. Does it matter operationally? It's the difference between a parity test that passes exactly and one that passes "within tolerance", and once you're in tolerance-land you've lost

the ability to detect a *real* divergence. Truncate both to microseconds and the two paths agree bit-for-bit.

The rewrite: bounded history

Here's the mistake, because it's more instructive than the fix.

Version one computed expanding statistics over each customer's entire history: lifetime transaction count, tenure since first seen, mean spend since signup, every merchant ever used. It used a Python loop with dictionaries tracking last-seen times, and it worked, and the leakage tests passed.

Then I wrote the serving parity test and it failed, and it was right to.

An authorisation request cannot carry an unbounded history. So a real service has two options: keep a stateful per-customer feature store — a second system that will drift from the training code — or silently compute something different from what the model was trained on. The second is the common failure and it's invisible.

So every history-derived feature is now bounded:

```
# src/data/features.py:147
DEFAULT_LOOKBACK = 31
```

The cost is real and I'm not going to pretend otherwise. We give up long-horizon signal: true tenure, lifetime distinct merchants, "this customer has never spent more than £50 in three years". On this data it costs little because the injected mechanisms are short-horizon. On a real portfolio a customer's multi-year baseline is genuinely informative, and the right answer there is a feature store sharing this exact code path — not a shorter window.

The rewrite also made it ~5x faster by replacing the Python loop with sliding windows, which was a happy accident rather than the goal.

The window trick

```
# src/data/features.py:167-181
def _window_view(values: np.ndarray, lookback: int, pad_value: Any) ->
  np.ndarray:
    """Matrix whose row ``i`` holds the ``lookback`` entries before
      position ``i``."""
    padded = np.concatenate([np.full(lookback, pad_value,
      dtype=values.dtype), values])
    view = np.lib.stride_tricks.sliding_window_view(padded, lookback)
    return view[: len(values)]
```

`sliding_window_view` gives you an `(n, k)` view over a 1-D array with no copy — it's stride manipulation. Prepending `lookback` pad values means row `i` of the view is `padded[i : i+k]`, which is original indices `i-k … i-1`. Exactly the `lookback` entries *before* `i`, with padding where history runs out.

The off-by-one to get wrong here: if you pad with `lookback - 1` instead of `lookback`, row `i` becomes `values[i-k+1 : i+1]` — which *includes the row itself*. Every feature then sees the transaction it's describing. Your amount z-score compares the amount to a mean that includes it, damping it toward zero. Your "distinct merchants" count includes the current merchant, so `is_new` is never true.

Would you catch it? The features would still be *shaped* right, still finite, still correlated with fraud. PR-AUC would move by a bit. And this is the bad part: including the current row makes the z-score *smaller* for anomalies, so it would make the model slightly worse, not better — which means you'd notice it eventually. The mirror-image error, using `lookback + 1`, silently drops the oldest slot and nobody ever notices.

The trailing slice `view[: len(values)]` matters too: `sliding_window_view` over an array of length `n + k` yields `n + 1` windows. Without the slice, you'd get one extra row and every downstream assignment would misalign by shape, which at least fails loudly.

The line that enforces causality

```
# src/data/features.py:262
    valid = np.isfinite(win_t) & (win_t < ts_sec[:, None])
```

This is the load-bearing line of the whole file. Two conditions:

`np.isfinite(win_t)` — window slots padded with `-inf` aren't real transactions.

`win_t < ts_sec[:, None]` — **strictly** less than. Not `<=`.

Why strict matters: two transactions can share a timestamp. With `<=`, a transaction would count its own simultaneous twin as history — and in card testing, bursts are exactly where near-simultaneous transactions cluster. The model would get information from a transaction that, in a real system, is being authorised in a different thread at the same moment.

The tempting wrong version is `win_t <= ts_sec[:, None]`, and it's tempting because it "includes more history" and your velocity counts go up, which makes the burst detection *better*. That's what makes it dangerous: the wrong version scores higher.

Note also that this check is elementwise on the window matrix, not a slice boundary. A slice-based version — "take everything before index i" — is correct only if the array is sorted, and stays correct only as long as nobody introduces a tie. This version is correct by construction.

Amount z-score: two defensive constants

```
# src/data/features.py:282-286
    prior_mean = np.where(has_hist, prior_mean, log_amount)
    # A floor on the denominator stops a customer with two
      near-identical
    # transactions from producing z-scores in the hundreds.
    amount_z = (log_amount - prior_mean) / (prior_std + 0.35)
    amount_vs_mean = log_amount - prior_mean
```

Line 282: with no history, set the prior mean to the *current* amount, making the z-score exactly zero. The alternative — leaving it at 0.0 — makes a customer's first transaction look like a massive positive anomaly, because `log1p(50) - 0` is large. Every customer's first transaction would then be flagged. That's not a subtle bug but it is an easy one, and it appears specifically in the rows you're least likely to eyeball.

Line 285: the `+ 0.35` floor on the denominator. A customer with two prior transactions of £20.00 and £20.01 has a prior standard deviation near zero, so a £25 transaction produces a z-score in the hundreds. That single row then dominates the batch's gradient. `0.35` is roughly the standard deviation of `log1p(amount)` for a typical customer, so it says "never be more confident than one typical customer's worth of spread". Raise it and the feature goes numb; remove it and you get infinities.

The clip in `scale()` (±10σ) is the second line of defence for the same problem. Belt and braces, because this is the failure mode that produces NaN gradients forty minutes into a training run.

Entity novelty, vectorised

```
# src/data/features.py:313-320
    def entity(keys: np.ndarray) -> tuple[np.ndarray, np.ndarray,
      np.ndarray, np.ndarray]:
        """Return (is_new, seconds since last seen, times seen, window
          codes)."""
        codes = pd.factorize(keys)[0].astype(np.int64)
        win_codes = _window_view(codes, k, -1)
        match = (win_codes == codes[:, None]) & valid
        seen = match.any(axis=1)
        last_seen = np.where(match, win_t, -np.inf).max(axis=1)
        since = np.where(seen, ts_sec - last_seen, NEVER_SECS)
        return (~seen).astype(np.float64), since,
          match.sum(axis=1).astype(np.float64), win_codes
```

`pd.factorize` turns arbitrary keys into integers so the comparison is a fast integer op rather than object comparison. The codes are **local to this array** — merchant "M001" might be code 3 for one customer and

code 7 for another. That's fine because every comparison is within a single call, and it's what lets the serving path (which sees a different subset) produce identical results.

The `& valid` on line 317 is doing real work: without it, a match against a padded slot would count. We pad codes with `-1` so a real code can't collide — but `pd.factorize` returns `-1` for NaN, so if a key were ever NaN this would silently break. Our keys are strings and never null, but that's an invariant held by convention rather than assertion. I'd add a check if this saw real data.

`NEVER_SECS = 3.2e8` (about ten years) rather than NaN or infinity: it keeps the tensors dense, lets a tree split on it cleanly, and survives `log1p` to a finite number. NaN would propagate through the standardiser and poison a whole feature column.

Out-of-fold target encoding, and the leak the tests caught

Risk encodings are the only features that consume labels, so they get special handling. The scheme is: block `k` is encoded using only blocks `< k`.

```
# src/data/features.py:601-602
        ranks = train[S.TIMESTAMP].rank(method="first").to_numpy()
        block = np.floor((ranks - 1) / max(len(train) / self.n_blocks,
          1)).astype(int)
```

`rank(method="first")` breaks ties by order of appearance, giving a total order. The default `method="average"` gives *fractional* ranks for ties, and `np.floor` on a fractional rank puts tied rows in whichever block the arithmetic lands in — nondeterministically with respect to row order.

```
# src/data/features.py:497-501
        prior_b = seen_pos / seen_n if seen_n > 0 else initial_prior
        k_arr = keys[mask]
        s = np.array([sum_y.get(k, 0.0) for k in k_arr],
          dtype=np.float64)
        c = np.array([cnt.get(k, 0.0) for k in k_arr], dtype=np.float64)
        out[mask] = (s + smoothing * prior_b) / (c + smoothing)
```

Here's the part worth stopping on. **The leakage test suite caught a real bug in this function on its first run, and the bug was in the prior.**

Version one smoothed towards `self.prior` — the fraud rate over the *whole* training window. Think about what that means for a row in block 0: its encoding is smoothed towards a number computed partly from labels in blocks 1–4, which arrive *after* it.

Is it a big leak? No. It's one scalar summarising ~1,100 labels; the influence of any single future label is about 1/1100. You would never see it in a metric.

Is it a leak? Yes. And `test_training_risk_encoding_is_out_of_time_within_train` flips the labels of the last training block and asserts the first block's encodings don't move. Under version one, they moved.

The fix is `prior_b`: block `k` smooths towards the rate observed over blocks `< k`, and block 0 smooths towards `initial_prior = 0.005` — a number fixed before looking at the data at all.

The `smoothing = 50.0` pseudo-count is the usual bias/variance dial. A merchant with 5 transactions and 1 fraud has an empirical rate of 20%; with smoothing 50 it reports `(1 + 50×0.005)/(5 + 50) ≈ 2.3%`. Without smoothing, every rare merchant that happened to catch one fraud becomes a maximally-suspicious merchant, and the model learns merchant IDs by heart.

The serving path

```
# src/data/features.py:773
            numeric[:, i] = causal[name] if name in causal else
              population[name]
```

`NUMERIC_FEATURES` is `CAUSAL_FEATURES + POPULATION_FEATURES`, and this line fills the matrix in that exact order by looking each name up in whichever dict has it. Column order is part of the contract with the model — the scaler's mean vector and the model's input layer are both indexed by it — so building this by `np.column_stack` of two dicts' `.values()` would work until someone reordered a dict literal.

What I'd change

The four risk encodings are the one place where the online and offline paths *don't* agree exactly, and it's inherent: during training they're out-of-fold, and an out-of-fold encoding is a property of the training procedure, not of the transaction. For any window lying after the training period — every window a deployed model actually sees — all 42 features match. Windows straddling the boundary differ in those four columns. Both facts are tested. It's the right trade but it's a wart, and if I were doing it again I'd consider dropping target encoding entirely in favour of frequency encoding plus letting the embeddings learn merchant risk, which would make parity unconditional.

Part 5: windows — `src/data/sequences.py`

Why separate. See above — different reasons to change. Also this file imports torch and `features.py` doesn't, which keeps the feature code usable in a context with no deep learning framework installed.

The lexsort trap

```
# src/data/sequences.py:61-66
    codes, _ = pd.factorize(customer_ids)
    # Stable sort by (customer, time) gives contiguous, chronologically
      ordered
    # blocks per customer while preserving the caller's tie-breaking.
    order = np.lexsort((timestamps, codes))
    sorted_codes = codes[order]
    boundaries = np.flatnonzero(np.diff(sorted_codes)) + 1
```

`np.lexsort` sorts by the **last** key first. So `(timestamps, codes)` means "primary key = codes, secondary = timestamps". That reads backwards to everyone the first time.

The tempting wrong version is `np.lexsort((codes, timestamps))`, which sorts primarily by *time* and secondarily by customer. Then `np.diff(sorted_codes)` finds a boundary essentially everywhere, and `np.split` produces thousands of one-element groups. Each customer's "history" becomes a single transaction.

Here's what makes it nasty: it doesn't crash. Every window is `[PAD, PAD, …, i]`, the mask is valid, the model trains, and it learns a per-transaction classifier. You get a number. It's just not the number you think you're measuring, and it's *lower*, so you conclude sequence models don't help.

Building the windows

```
# src/data/sequences.py:68-73
    pad = np.full(max_len - 1, PAD_ROW, dtype=np.int64)
    for group in np.split(order, boundaries):
        padded = np.concatenate([pad, group])
        view = np.lib.stride_tricks.sliding_window_view(padded, max_len)
        windows[group] = view
    return windows
```

Same trick as `_window_view`, one difference: we pad by `max_len - 1`, not `max_len`, because here the window is *inclusive* of the scored transaction. Row `i`'s window is `[i-31 … i]`, 32 slots, with the scored transaction last.

That's the second off-by-one in this codebase and it's the opposite of the first. In `_window_view` we want the rows strictly before `i`, so we pad by `k`. Here we want `i` included, so we pad by `k-1`. Two functions, three lines apart in spirit, opposite constants. If you're rebuilding this, write the test before the function.

Why left-padded and not right-padded. Because it means position `L-1` is *always* the transaction being scored, for every row, regardless of history length. The TCN reads `h[:, :, -1]`. The Transformer reads position `-1`. Neither needs to know how much history this customer has. Right-padding would make the prediction position vary per row and every architecture would need a gather.

The gather

```
# src/data/sequences.py:175-182
    win = data.windows[rows]
    valid = win >= 0
    safe = np.where(valid, win, 0)

    numeric = data.numeric[safe]
    numeric[~valid] = 0.0
    categorical = data.categorical[safe]
    categorical[~valid] = PAD_INDEX
```

`safe` replaces `-1` with `0` before indexing, because numpy would happily interpret `-1` as "last row" and silently give you a real transaction from the end of the dataset. Then we zero those slots out.

The critical detail: `data.numeric[safe]` is **fancy indexing**, which returns a copy. So `numeric[~valid] = 0.0` mutates the copy. If you refactored this to basic slicing — which returns a *view* — you'd be zeroing rows of the backing array, corrupting the dataset progressively as training proceeded. Early batches fine, later batches increasingly garbage, loss curve slowly rising. Good luck.

Weighted sampling

```
# src/data/sequences.py:245-246
            p = self.sample_weights / self.sample_weights.sum()
            order = self.rng.choice(self.rows, size=len(self.rows),
              replace=True, p=p)
```

`replace=True` is not optional. Sampling `len(rows)` items without replacement from `len(rows)` items is just a permutation, and the weights do nothing. That's a genuinely invisible bug: your "weighted sampling" arm trains fine, produces sensible numbers, and is identical to the unweighted arm. You'd conclude that resampling has no effect on calibration and write it up.

The test that catches it (`test_weighted_sampling_oversamples_positives`) draws an epoch and asserts the positive rate is at least 5× the natural rate.

Part 6: the leakage tests — write these now, not at the end

Why here. We have features and windows and no models. This is exactly the right moment, because the tests are about the data pipeline and nothing else, and because every model we build from here on will be evaluated with a metric we can't yet trust.

Why they're behavioural, not structural. We could write a test that reads `features.py` and asserts it never indexes past `i`. That test breaks on every refactor and passes on any leak introduced through a library call. Instead: **perturb the future, assert the past doesn't move.**

```
# tests/test_leakage.py:74 (the core one)
@pytest.mark.parametrize("quantile", [0.3, 0.6, 0.85])
def test_causal_features_ignore_future_transactions(
    transactions: pd.DataFrame, quantile: float
) -> None:
    """Scrambling every transaction after t must not move any feature
      before t."""
```

It scrambles amount, merchant, MCC, device, device type, country, channel and label for every transaction after a cut point, recomputes all features, and asserts bit-identical equality (`rtol=0, atol=0`) for every row before the cut.

That catches leakage regardless of mechanism — including leakage introduced later by someone who's never read this file.

The companion tests each catch something the others can't:

- `test_causal_features_ignore_deleted_future_rows` (line 93) *truncates* the future instead of perturbing it. Perturbation catches value leakage; truncation catches leakage through *counts*. A rolling window that accidentally looks forward changes size when later rows disappear but not when their values change.
- `test_labels_never_enter_causal_features` (line 112) flips every label and asserts every causal feature is unchanged. This is Fact three, mechanised.
- `test_single_customer_features_are_prefix_stable` (line 128) computes features for the first half of a customer's history and asserts they equal the first half of the full computation. This is the serving contract in miniature.
- `test_windows_contain_only_own_customer_and_past` (line 162) checks the window index array directly: every slot is the same customer, no slot is later than the scored row, the scored row is last, and indices increase.

And the one that keeps the others honest:

```
# tests/test_leakage.py:290-303 (abridged)
def test_leakage_detector_has_teeth(transactions: pd.DataFrame) -> None:
    """The future-perturbation check must reject a known-leaky
      feature."""
    ...
    differs = not np.allclose(
        base.loc[past, "leaky_amount_z"].to_numpy(),
        perturbed.loc[past, "leaky_amount_z"].to_numpy(),
        equal_nan=True,
    )
    assert differs, "the leakage check failed to notice a feature that
      reads the future"
```

It builds a deliberately leaky feature — the classic `groupby.transform("mean")` where an expanding mean belongs — and asserts the check rejects it.

Without this, a refactor that made the assertions vacuous (say, comparing an empty column selection) would leave you with a green suite and no protection. Every assertion-heavy test file should have one of these.

This suite paid for itself on its first run by catching the target-encoding prior. Not a bug I'd have found by reading.

Part 7: the baseline, and how we'll judge everything

`src/evaluation.py` — before the models, because we can't read results without it

Why it exists as a module rather than a few calls to sklearn. Because the metric definitions *are* decisions, and they need to be made once, in one place, with the reasoning attached. Three of them in particular.

Decision one: accuracy is reported only next to its own refutation.

```
# src/evaluation.py:244-258 (abridged)
    in_unit_interval = bool(np.all((scores >= 0) & (scores <= 1)))
    brier = float(np.mean((scores - y_true) ** 2)) if in_unit_interval
      else float("nan")
    predicted = (scores >= 0.5).astype(np.float64) if in_unit_interval
      else np.zeros_like(y_true)
    ...
        accuracy_of_always_legit=float((y_true == 0).mean()),
```

Every result object carries `accuracy` *and* `accuracy_of_always_legit`. On our test set both are about 99.48%. Stating "accuracy is inappropriate for imbalanced problems" is an assertion; printing two numbers that are equal to two decimal places is a demonstration, and it takes one extra field.

The `in_unit_interval` guard exists because we also evaluate raw logits sometimes, and a Brier score on logits is meaningless — better `nan` than a plausible-looking wrong number.

Decision two: precision at capacity takes exactly k rows by rank.

```
# src/evaluation.py:57, 83-84
    k = max(int(np.ceil(capacity * n)), 1)
    ...
    order = np.argsort(-scores, kind="stable")[:k]
    caught = float(y_true[order].sum())
```

The tempting version is to compute a threshold and count everything above it:

```
threshold = np.quantile(scores, 1 - capacity)   # tempting
flagged = scores >= threshold                    # and wrong
```

Tree ensembles produce enormous numbers of *identical* scores — LightGBM's output is a sum over a finite set of leaf values. At a 0.1% capacity on 72,000 rows you want 73 alerts. If the threshold lands on a tied clump you get 800. Your reported "precision at 0.1% capacity" is then precision at 1.1% capacity, which is a different and much easier

number.

Taking the top `k` by rank gives you exactly the alert volume you promised. The `kind="stable"` makes tie-breaking deterministic across runs, so the number doesn't wobble between seeds for reasons unrelated to the model.

`test_alert_volume_is_exact_under_ties` in `tests/test_evaluation.py` constructs 1000 rows where 995 share a score and asserts `n_alerts == 50` at 5% capacity.

Decision three: per-mechanism recall, and refusing to fake it.

`recall_by_fraud_type` omits mechanisms absent from the data rather than reporting them as zero recall, and `evaluate` suppresses the whole breakdown when every positive is `UNKNOWN` — which is what the real-data loaders produce. A table of zeros looks like a model failure; an absent table correctly says "this dataset can't answer that".

```
# src/evaluation.py:227
    if y_true.sum() == 0:
        raise ValueError("slice contains no positives; metrics would be
          undefined")
```

This one *crashes* rather than returning NaN. It's a programming error — you've sliced your data wrong — and silently returning NaN means it propagates into a results table as a blank cell that someone reads as "not applicable".

`src/baselines.py` — the number to beat, built to win

Why it exists. If we compare three neural architectures to each other, we've learned nothing about whether any of them should exist. The baseline is the control.

Why it has to be *strong*. A weak baseline is obvious to any reader and invalidates the entire project. The usual tells are default hyperparameters, no categorical handling, and no velocity features. So

the baseline gets everything the sequence models get — the same 42 features, flattened to one row — plus native categorical handling:

```
# src/baselines.py:163-166
    train_set = lgb.Dataset(x_train, label=y_train,
      categorical_feature=spec.categorical_columns)
    val_set = lgb.Dataset(
        x_val, label=y_val, reference=train_set,
          categorical_feature=spec.categorical_columns
    )
```

That `categorical_feature` argument is the fair-fight clause. LightGBM's categorical splits partition categories into two sets by sorted gradient statistics — genuinely competitive with a learned embedding, and much cheaper. Omitting it, so merchant codes are treated as ordinal integers, would hand the comparison to the neural nets on a technicality. The `reference=train_set` on the validation set makes it reuse the training histogram bins; without it LightGBM bins the validation data independently and the two disagree.

```
# src/baselines.py:62
    "metric": "average_precision",
```

Early stopping on average precision, not on the default binary log-loss. At a 0.6% positive rate the loss keeps improving long after ranking has stopped, so a log-loss-stopped model trains further and ranks worse. Same decision as for the neural nets — applied to both sides, which is the point.

```
# src/baselines.py:200
    n_leaves = int(booster.num_trees() * params["num_leaves"])
```

We report this as the baseline's "size". A parameter count for a tree ensemble is a category error; total leaf count is the honest analogue. It's flagged as such in the README table, because otherwise a reader compares 59,031 against the GRU's 78,031 and concludes they're comparable objects.

The `rule_baseline` function is four hand-written conditions — large amount for this customer, burst in the last hour, unfamiliar device, unfamiliar merchant. It scores PR-AUC 0.047. That number is worth having: it tells you how much of the achievable signal *isn't* available to obvious rules, and therefore how much the learned models are actually contributing.

`src/artifacts.py` — the boundary between training and everything else

Why it exists. Calibration, policy, monitoring and the write-up all need the same thing: the scores a model assigned to validation and test rows. Without a shared container, each of them either retrains the model or reaches into a trainer's internals.

```
# src/artifacts.py (dataclass fields)
    name: str
    family: str
    seed: int
    val_scores: np.ndarray
    test_scores: np.ndarray
    train_seconds: float
    n_params: int
    meta: dict[str, Any]
```

That's the entire contract between "training" and "analysis". Everything after training reads only this. It's also why the full grid can run overnight and the analysis can be re-run in 90 seconds — which mattered enormously when I found the ensembling bug, because fixing it needed no retraining at all.

It's saved as `.npz` with a JSON header string. Not pickle, because these files outlive the code that wrote them and pickle would tie them to the class definition.

Part 8: the models — `src/models/`

Why a package and not one file. The shared encoder is genuinely shared, and the three architectures differ only in how they reduce a sequence to a vector. Separate files make it obvious how little architecture-specific code there is.

`src/models/base.py` — the contract every architecture signs

```
# src/models/base.py:32-43
def embedding_dim(cardinality: int, max_dim: int = 16) -> int:
    rule = int(round(1.6 * cardinality**0.56))
    return int(max(2, min(max_dim, rule)))
```

The `1.6 · cardinality^0.56` heuristic is the fastai rule. The interesting part is `max_dim`. Our merchant vocabulary is ~2,910, and the uncapped rule asks for 139 dimensions — a 400,000-parameter table fitted from roughly 1,100 positive training examples. That table would memorise merchant IDs.

Capping at 16 gives 46,560 parameters for merchants, which is *still* 60% of the GRU's total. The README says so explicitly, because a reader comparing "78k parameters" to a GBM deserves to know most of it is a lookup table rather than computation.

```
# src/models/base.py:103
                nn.Embedding(spec.cardinality, dim,
                  padding_idx=PAD_INDEX)
```

`padding_idx=PAD_INDEX` does two things: initialises row 0 to zeros, and — the important one — excludes it from gradient updates. Without it, the padding embedding drifts during training, so a padded slot contributes a learned nonzero vector, and how much depends on how often padding appeared. Sequences with different history lengths get systematically different treatment.

```
# src/models/base.py:137
        return x * mask.unsqueeze(-1)
```

The encoder zeroes padded positions on the way out. Belt and braces on top of `padding_idx`, because the encoder also has a `LayerNorm` and a `Linear` with a *bias* — and a bias applied to a zero input is not zero. Without this line, every padded position emits the projection layer's bias vector, and a convolution sliding over it picks that up as signal.

This is the trap I'd most expect someone to hit rebuilding this. It's invisible: the model trains, the loss goes down, and short-history customers get a systematically different representation than long-history ones. The test that catches it (`test_padding_does_not_affect_the_score`) sets padded slots to 13.7 and asserts every score is bit-identical.

Then the piece of geometry that the RNN needs:

```
# src/models/base.py:277-282
    b, length, _ = x.shape
    lengths = mask.sum(dim=1).long().clamp(min=1)
    pad = length - lengths
    index = (torch.arange(length, device=x.device).unsqueeze(0) +
      pad.unsqueeze(1)) % length
    rolled = torch.gather(x, 1, index.unsqueeze(-1).expand(b, length,
      x.size(-1)))
    return rolled, lengths
```

Our windows are left-padded because the convolutional and attention models want the scored transaction at a fixed final position. A recurrent model wants the opposite: it shouldn't consume a variable-length run of zeros before seeing any data. So we roll each row left by its own padding count.

The modular arithmetic: for row with `pad = p`, output position `j` reads input `(j + p) mod L`. For `j < L - p` that's a real position; for `j >= L - p` it wraps into the pad region. Real data first, padding trailing.

The tempting alternative is `pack_padded_sequence`, which is what PyTorch offers and what most code uses. It's exact and it's fast — and it does not export to ONNX as a static graph, which would sink the serving story. The `% length` version is pure gather ops that trace cleanly.

`clamp(min=1)` guards a length-zero row. Can't happen — the scored transaction is always real, so `mask[:, -1]` is always 1 — but a zero here would make `gather_at(outputs, lengths - 1)` index `-1` and silently return the *last* timestep, which for an all-padding row is garbage that looks like a valid number.

`src/models/tcn.py` — causality by construction

```
# src/models/tcn.py:43, 61
        self.pad = (kernel_size - 1) * dilation
    ...
        return conv(nn.functional.pad(x, (self.pad, 0)))
```

`(self.pad, 0)` pads `(left, right)`. Left only. Then the convolution's output is longer than its input by exactly `self.pad`, and since the padding was on the left, output position `t` depends on inputs `t - pad` … `t`. All in the past.

The tempting wrong version is `padding=self.pad` as a Conv1d argument, which pads **both** sides symmetrically. Output position `t` then depends on inputs up to `t + pad`. Your TCN reads the future.

And this is the worst kind of bug in this codebase, because the shape is identical, no assertion fires, and **your metrics improve**. A model that can see the next transaction predicts card testing beautifully. You'd ship it.

What makes the sliced-causal-pad version defensible is that it's structural: even if the window construction were wrong, position `t` cannot reach `t+1`.

```
# src/models/tcn.py:113
        return 1 + sum(2 * (k - 1) * (2**i) for i in
          range(self.config.n_layers))
```

Receptive field: two convolutions per block, dilations doubling. Four blocks with kernel 3 reach 61 positions, covering our 32-window. Two blocks reach 13 — the model would be structurally unable to see two-thirds of the history the other architectures get, and you'd conclude TCNs are bad at this task when actually you built a small one. There's a warning at construction if the receptive field is short.

```
# src/models/tcn.py:128
            h = block(h) * mask_c
```

Re-masking after *every* block. One block's convolution smears real values into positions that were padding — that's what convolution does — so without re-masking, layer 2 sees "data" where layer 1 saw padding.

`src/models/transformer.py` — the mask, and the NaN nobody expects

```
# src/models/transformer.py:173-181
        positions = torch.arange(length, device=mask.device)
        query = positions.unsqueeze(1)
        key = positions.unsqueeze(0)
        causal = key > query
        diagonal = key == query

        pad_keys = mask.unsqueeze(1) < 0.5  # (B, 1, L)
        blocked = causal.unsqueeze(0) | pad_keys
        return blocked & ~diagonal.unsqueeze(0)
```

Two things here.

First, why index comparisons instead of `torch.triu` and `torch.eye`. Because they export to ONNX operators that ONNX Runtime's CPU provider doesn't implement. `torch.eye` becomes `EyeLike`, and loading the graph fails with `NOT_IMPLEMENTED`. The service catches that exception and falls back to PyTorch — so everything *works*, and you

quietly lose your 2.8× speedup and never notice unless you check which runtime is live. That's how it was for a while.

Second, and more subtle: line 181. Combine a causal mask with a padding mask and a padded *query* position has nothing left to attend to — every key at or before it is either future or padding. Softmax over a row of all `-inf` is NaN. The NaN propagates through the residual stream and your loss is NaN from step one.

Forcing the diagonal open means every position can always attend to itself. Padded positions produce garbage, but we never read them, and because padded slots are masked as *keys* the garbage can't reach the scored position.

The tempting wrong fix is to clamp the softmax or add epsilon to the denominator. That removes the NaN and leaves the padded rows with an arbitrary distribution that *does* feed forward.

```
# src/models/transformer.py:85
                q, k, v, attn_mask=~blocked.unsqueeze(1)
```

Note the `~`. `scaled_dot_product_attention`'s boolean `attn_mask` uses `True` = "participate", the opposite convention to the `-inf` fill. Get it backwards and the model attends to exactly the positions it shouldn't. There's a test asserting the fused and explicit paths compute the same function, precisely because this convention flip is so easy to get wrong.

Part 9: training — `src/train.py`

```
# src/train.py:86-90
    bce = nn.functional.binary_cross_entropy_with_logits(logits,
      targets, reduction="none")
    prob = torch.sigmoid(logits)
    p_t = prob * targets + (1 - prob) * (1 - targets)
    alpha_t = alpha * targets + (1 - alpha) * (1 - targets)
    return (alpha_t * (1 - p_t).pow(gamma) * bce).mean()
```

`binary_cross_entropy_with_logits` rather than `sigmoid` then `binary_cross_entropy` — the fused version is numerically stable via log-sum-exp. The separate version overflows for logits around ±80, which happens once a model gets confident.

`p_t` is "probability assigned to the correct class". `(1 - p_t)^gamma` is the focusing term: a well-classified example with `p_t = 0.99` and `gamma = 2` gets weighted 0.0001.

`alpha_t` is where people go wrong. With `alpha = 0.25`, positives get 0.25 and negatives 0.75 — the rare class is weighted *down*. That looks backwards and isn't: the focusing term already up-weights hard examples, which are mostly positives, and alpha counteracts the overcorrection. We measured the consequence later and it's real: focal-loss models are systematically **under**-confident in the alert region (predicting 0.570 where the true rate is 0.824).

```
# src/train.py:279, 296-300
        pr_auc = float(average_precision_score(y_val, val_scores))
        ...
        if pr_auc > best_pr:
            best_pr = pr_auc
            best_state = copy.deepcopy(model.state_dict())
            best_epoch = epoch
        elif epoch - best_epoch >= cfg.patience:
```

Early stopping on validation PR-AUC. `copy.deepcopy` on the state dict is not optional — `state_dict()` returns tensors that *share storage* with the live model, so without the deepcopy your "best" checkpoint is a set of references that keep changing as training continues. You'd restore the final weights, not the best ones, and only notice because your restored model doesn't reproduce its own validation score.

The `elif` is deliberate: patience is counted from the last *improvement*, and the branch only runs on a non-improving epoch.

```
# src/train.py:247
        drop_last=True,
```

Training only. With 512-row batches and a 0.6% positive rate, a final batch of 30 rows probably contains zero positives, and under focal loss that's a batch of pure negative gradient. Dropping it costs a fraction of an epoch. For evaluation we keep every row, obviously.

Part 10: calibration — `src/calibration.py`

Why it's a module. Because the policy in the next file thresholds an *absolute probability*, and that arithmetic is only valid if the probability means what it says. Ranking isn't enough.

```
# src/calibration.py:182
        edges = np.unique(np.quantile(probabilities, quantiles))
```

Quantile bins, not equal-width. This is the difference between measuring calibration and not. At a 0.6% base rate, over 99% of predictions are below 0.05. Equal-width bins put essentially the entire dataset in bin one and report a near-zero ECE for any model, however wrong.

`np.unique` collapses duplicate edges, which happens constantly here because many quantiles of a spiky score distribution coincide. Without it you get zero-width bins and division by zero.

But quantile bins aren't enough either, and this is the sharpest thing I learned in the whole project:

```
# src/calibration.py:117-119
    k = max(int(np.ceil(capacity * n)), 1)
    top = np.argsort(-np.asarray(probabilities), kind="stable")[:k]
    return float(np.mean(probabilities[top])),
      float(np.mean(np.asarray(y_true)[top]))
```

`alert_region_calibration` restricts to the top 0.5% — the alerts a team actually works. **Aggregate calibration is dominated by the 99%+ of transactions correctly scored near zero.** A model can look immaculate overall and be badly wrong in the only band where a decision is taken.

This wasn't in the first version. I added it because the aggregate numbers were telling the *opposite* story to the truth: `gru_focal`'s aggregate mean prediction was 2.0× the true rate (looks overconfident), while in the alert region it was 0.69× (actually under-confident). Focal loss flattens the distribution from both ends — it raises the median prediction from 0.00002 to 0.004 while lowering the top. One number told the wrong story with complete confidence.

```
# src/calibration.py:253, 265
        phi = (np.sqrt(5.0) - 1.0) / 2.0
        ...
        self.temperature = float(np.exp((lo + hi) / 2.0))
```

Golden-section search over `log T` rather than gradient descent. The objective is one-dimensional and unimodal, so this converges in 60 evaluations with no optimiser, no learning rate, and no dependency. Searching over `log T` rather than `T` keeps the temperature positive by construction — a negative temperature would invert the ranking, which is the sort of thing an unconstrained optimiser will find if the data is odd enough.

Part 11: cost — `src/policy.py`

This is the headline, and it's also where I made the biggest mistake in the project. Both the mechanism and the mistake are worth your attention.

```
# src/policy.py:110, 120-131
    blocked = scores >= threshold
    ...
    total = (
        cost.false_positive_cost * float(fp.sum())
        + cost.false_negative_cost_fixed * float(fn.sum())
        + cost.false_negative_amount_multiplier * missed_value
        + cost.review_cost * float(tp.sum())
        + (1.0 - cost.true_positive_recovery) * blocked_value
    )
```

`missed_value` is `amounts[fn].sum()` — the actual currency value of fraud that got through. A flat per-fraud cost would hide the thing that matters most: the expensive frauds are large, and a model can be excellent at catching cheap ones while losing more money than a worse model.

The last term was added during this walkthrough. `true_positive_recovery` was documented in `CostConfig` as a real knob and was never referenced. A reader would turn it down and see nothing change. At the default of 1.0 the term is exactly zero, so no reported number moved — the knob just does something now.

```
# src/policy.py:174
    quantiles = np.unique(
        np.quantile(scores, np.linspace(0.0, 1.0, n_points),
          method="linear")
    )
```

Threshold candidates are score *quantiles*, not a uniform grid over [0,1]. With scores massed near zero, a uniform grid spends 390 of its 400 points in a region where no decision changes and resolves the interesting region with about ten.

```
# src/policy.py:214
    best = eligible.loc[eligible["total_cost"].idxmin()]
```

And the rule that makes this honest: this is called on **validation** scores, and the resulting threshold is applied unchanged to test. Tuning the threshold on test is a second fitting step that reports its own training error. We also report the gap to the unattainable oracle threshold, so the size of that effect is visible rather than assumed small.

The mistake

The first version of `experiment.py` fitted one calibrator on the **mean of three seeds' scores** and thresholded that. Averaging three models' predictions is an ensemble. And ensembling helps high-variance models much more than low-variance ones — the sequence models' seeds differ a lot, LightGBM's barely differ at all.

Reported result: sequence models 27% cheaper than LightGBM.

Nothing leaked. Nothing was mislabelled. The comparison was even *fair*, because both sides were ensembled. It was just answering a question nobody asked, because **what this repository deploys is one checkpoint, not an ensemble of three.**

Costing per seed and aggregating like every other metric:

	ensemble (wrong question)	per-seed (what ships)
best sequence arm	0.0665	0.0799 ± 0.0122
LightGBM	0.0906	0.0860 ± 0.0035
gap	–27%	–7%, inside one σ

The individual seeds cost 0.0711, 0.0749, 0.0938 for `gru_focal` against 0.0821, 0.0869, 0.0890 for LightGBM. Those distributions overlap. **On three seeds you cannot claim a cost saving.**

The ranking advantage survived — PR-AUC 0.870 ± 0.014 vs 0.826 ± 0.001 is about three standard deviations and it's real. The cost advantage didn't. That asymmetry is itself the finding: PR-AUC integrates over every threshold, while expected cost is evaluated at *one*

operating point chosen on validation from ~320 frauds, and that choice is noisy.

If you're rebuilding this: **aggregate every stage the same way, and decide early whether you're evaluating an ensemble or a model.** The bug is invisible because both answers are defensible in isolation.

Part 12: monitoring — `src/monitoring.py`

Why it exists. A fraud model that has silently stopped working is worse than no model: it produces confident approvals for attacks it can no longer see. And because our simulator injects adversarial drift, we can *test* the monitor rather than assert it works.

```
# src/monitoring.py:86
    edges[0] = -np.inf
```

PSI bins are derived from the **reference** period and then held fixed. Forcing the outer edges to infinity means a window containing scores outside the reference range still gets binned rather than dropped.

The tempting wrong version is to re-derive quantile bins per window. Then every window is compared against its own quantiles, every window looks perfectly stable by construction, and your drift monitor never fires. That is how a monitor ends up being decoration, and it's a one-line difference from a working one.
`test_psi_uses_fixed_edges_across_windows` pins it down.

```
# src/monitoring.py:119-121
    ref_frac = np.maximum(ref_counts / max(ref_counts.sum(), 1), _EPS)
    cur_frac = np.maximum(cur_counts / max(cur_counts.sum(), 1), _EPS)
    return float(np.sum((cur_frac - ref_frac) * np.log(cur_frac /
      ref_frac)))
```

The `np.maximum(..., _EPS)` floor: an empty bin gives `log(0/x) = -inf` and PSI becomes NaN or infinite. Empty bins are guaranteed here — a fraud score distribution has bins that some weeks simply don't populate.

The negative result, which is the interesting part

Here's what the standard playbook says: monitor PSI on the score distribution. Here's what happened.

Score PSI peaked at 0.0011 across the entire test period. The conventional warning threshold is 0.10. It never came close. It never fired. Meanwhile precision at fixed capacity fell from 1.00 to 0.64 in the same window.

It was never going to work, and the reason is structural: **more than 99% of transactions are legitimate and the adversary does not change their behaviour at all.** The distribution PSI measures is dominated by a population that didn't move. The fraud tail gets completely rewritten underneath it and the bulk barely twitches.

So the monitor grew a second signal:

```
# src/monitoring.py:159
    deviation = min(ratio, 1.0 / ratio)
```

`alert_rate_ratio` is the window's alert rate at the **fixed production threshold**, divided by its reference value. When fraud learns to score low, the alert rate at an unchanged threshold falls immediately — and it's just as label-free as PSI.

The `min(ratio, 1/ratio)` makes the check symmetric: a ratio of 0.5 and a ratio of 2.0 are both equally far from normal. Alerting only on drops would miss a spike, which is an operational problem in its own right because the review queue is a fixed size.

In the run: 0.95 → 0.80 → **0.57 (warn)** in the first window containing drifted traffic. It caught what PSI missed.

`test_score_psi_alone_is_a_weak_detector_here` asserts PSI stays below the warning threshold. That's a test asserting something *doesn't* work, which feels odd, but it means if a future change makes PSI effective the write-up gets revisited rather than quietly becoming wrong.

And the reason label-based precision can't be the alarm:

```
# src/monitoring.py — WindowReport carries both
        precision_at_capacity: ...
        alert_rate_ratio: ...
```

Precision needs confirmed chargebacks, which arrive weeks later. By the time it moves, the loss is booked. It confirms; it doesn't warn.

Part 13: interpretability — `src/explain.py`

Why it exists. An analyst working an alert needs to know what to look at. A model-risk reviewer needs to know what the model keyed on. Different questions, and neither is answered by "the score was 0.87".

The honest framing is the whole content of this module, so I'll state it the way the code does:

```
# src/explain.py (module constant)
ATTENTION_CAVEAT = (
    "Attention weights show which prior transactions the model read
      from, not "
    "why it decided. They are not a causal attribution: a low-weight
      position "
    "can still be decisive and a high-weight one can be inert. ..."
)
```

Attention is not explanation. A low-weight position can still be decisive — its information may already be carried in the residual stream. A

high-weight one can be inert, because attention multiplies values that may be near zero. Different attention distributions can produce identical outputs, so no single one is *the* explanation.

What it's genuinely good for is narrower and still useful: pointing an analyst at related prior activity. So every attention explanation ships alongside a gradient attribution, which answers a different, counterfactual-flavoured question — how much does the score move if this input moves.

```
# src/explain.py (in explain_sequence_model)
    gradient = numeric.grad
    if gradient is None:  # pragma: no cover - backward() always
      populates it
        raise RuntimeError("no gradient was produced for the numeric
          inputs")
    contribution = (gradient * numeric).detach()[0].numpy()
```

The explicit `None` check is for the type checker, but it also documents an assumption: this only works because we called `.requires_grad_(True)` on the input and ran `.backward()`. Refactor to `torch.no_grad()` for speed and this crashes rather than silently returning zeros.

The caveat is emitted in the module, in the worked example, and in the API response body. That last one matters: a reviewer reading the JSON contract sees the limitation without reading any docs.

Part 14: serving — `src/api/`

Why a package. Four concerns that change independently: the wire format (`schemas.py`), the scoring logic (`service.py`), the HTTP layer (`app.py`), and the audit trail (`audit.py`). And critically, the serving container should be able to install `schemas` + `service` + `app` without LightGBM, SHAP or matplotlib.

`src/api/service.py` — the arithmetic that makes parity work

```
# src/api/service.py:287-303
    @property
    def required_history(self) -> int:
        """Prior transactions a request should carry for exact parity
          with training.
        ...
        """
        return self.config.max_len - 1 + self.builder.lookback
```

This is 31 + 31 = 62, and it took me an embarrassingly long time to get right.

The model reads a window of 32 positions. **Every position in that window carries features built from its own 31 prior transactions.** So reproducing training-time inputs exactly needs 31 transactions to fill the window, plus 31 more behind the *oldest of them*.

The natural mistake — and the one I made — is to send `max_len - 1 = 31` prior transactions. Then the scored transaction's own features are perfect, but window position 0 sees zero history where it saw 31 during training, position 1 sees one, and so on. The features degrade smoothly toward the front of the window.

Why it's invisible: the scored transaction's features, which dominate the prediction, are *exactly right*. The score is close. It's just not the score the batch pipeline computes, and the discrepancy is largest for exactly the customers with the most history.

```
# src/api/service.py:320-335
        history = sorted(request.history, key=lambda t:
          t.timestamp)[-self.required_history :]
        records = [t.model_dump() for t in [*history,
          request.transaction]]
        numeric, categorical = self.builder.transform_window(records)
        # Features needed the deeper history; the model only sees the
          last L rows.
        numeric = numeric[-max_len:]
        categorical = categorical[-max_len:]
        ...
        padded_numeric[0, max_len - n :] = numeric
        ...
        mask[0, max_len - n :] = 1.0
```

Compute features over 63 rows, hand the model the last 32. The `[max_len - n :]` slicing puts the real rows at the **end**, matching the left-padding convention from `build_windows`. Get that backwards and every model reads padding as its scored transaction.

The `sorted(...)` is a real guard: a client can send history in any order. If the scored transaction is older than some history entry, `customer_causal_features` raises `ValueError` and the app turns it into a 400. That's the right failure — loud, attributable, and client-fixable.

```
# src/api/service.py:131-132
        opset_version=17,
        dynamo=False,
```

`dynamo=False` pins the legacy TorchScript-based exporter. The dynamo exporter produces a different graph and, at the time, a different set of unsupported ops. Pinning the exporter is the same instinct as pinning a dependency version.

```
# src/api/service.py:259
                options.intra_op_num_threads = (serving or
                  ServingConfig()).onnx_intra_op_threads
```

One thread. At batch size 1 there is nothing to parallelise across, and thread hand-off shows up directly in the tail latency the budget is written against. The default (all cores) is worse *and* consumes the whole box.

```
# src/api/service.py:353
        if self.session is not None:
```

The runtime falls back to PyTorch when ONNX is unavailable. That's a deliberate swallow — the service should stay up on a slower path rather than fail to boot — but it's exactly how the `EyeLike` problem hid for a while. If I were running this for real I'd add a startup assertion that the expected runtime is the live one, and let it crash loudly in production while still falling back in development.

`src/api/audit.py` — the record that has to exist before the response

```
# src/api/audit.py:108-112
        record.finalise()
        line = json.dumps(asdict(record), separators=(",", ":"),
          default=str)
        with self._lock, self.path.open("a", encoding="utf-8") as
          handle:
            handle.write(line + "\n")
            handle.flush()
```

Append mode, one JSON object per line, **flushed per record**. Buffering would lose the decisions taken in the seconds before a crash — exactly the ones anyone will ask about. It's a real cost per request and it's the right trade.

```
# src/api/audit.py:136 (in verify)
            digest = hashlib.sha256(
                json.dumps(record, sort_keys=True, separators=(",",
                  ":")).encode()
            ).hexdigest()
```

`sort_keys=True` and explicit `separators` in **both** the write and the verify path. Without them the digest depends on dict ordering and whitespace, so verification fails on records written by a different Python version.

Being straight about the limits: this detects *individual* record modification. It is not a tamper-evident chain — there's no hash linking record N to record N-1, so deleting a whole record is undetectable. A

real system chains them and ships them off-box. That's stated in DECISIONS.md rather than implied away.

`src/api/app.py` — which failures crash, which don't

```
# src/api/app.py:106
    except Exception as exc:  # pragma: no cover - exercised by the
      degraded-path test
        state.load_error = str(exc)
        LOGGER.warning("no model loaded: %s", exc)
```

A bare `except Exception` at startup, which normally I'd object to. Here it's deliberate: a missing model bundle should bring the process **up** in a degraded state, not crash-loop. `/health` reports `degraded`, `/score` returns 503 with the actual load error in the body, and an orchestrator sees a diagnosable service rather than a container restarting every ten seconds with the reason buried in logs.

The error-handling split across the whole service:

failure	behaviour	why
no model bundle at startup	log, come up degraded	diagnosable beats crash-loop
ONNX session fails to build	log, fall back to PyTorch	slower is better than down
history out of order	`ValueError` → HTTP 400	client's bug, client can fix it
malformed request body	pydantic → HTTP 422	free, and precise
unknown merchant/device	map to `UNK_INDEX`, score normally	expected, not exceptional
evaluating a slice with no positives	`raise ValueError`	programming error, must not return NaN
scaling before fitting	`raise RuntimeError`	ordering bug, fail immediately

The rule: **swallow infrastructure failures where a degraded path exists; crash on contract violations.** A missing ONNX runtime is infrastructure. Calling `scale()` before `fit_transform()` is a contract violation, and returning something plausible would corrupt every number downstream.

Part 15: orchestration — `src/experiment.py` and `src/report.py`

Why `experiment.py` exists. Because the alternative is a notebook, and a notebook lets the write-up drift from the code that produced it. One command produces every number in the README.

The ordering it enforces is the ordering of this document, and the step that's easy to get wrong is step 6:

```
# src/experiment.py (deployment selection, abridged)
    mean_val_cost = (
        pd.DataFrame([...]).groupby("model")["val_cost"].mean()
    )
    best_model = str(mean_val_cost.idxmin())
    # Among that arm's seeds, deploy the one whose validation cost is
      closest to
    # its arm's mean -- the representative run, not the flattering one.
    deploy_candidates = [r for r in policy_results if r.model ==
      best_model]
    best_policy = min(
        deploy_candidates,
        key=lambda r: abs(r.val.cost_per_transaction -
          mean_val_cost[best_model]),
    )
```

Two selections, both on **validation**. Which arm: lowest mean validation cost. Which seed within it: the one closest to that arm's mean.

That second one is the non-obvious bit. Picking the seed with the *lowest* validation cost selects on noise — the seed spread is comparable to the gap between arms, so you'd be shipping whichever run got lucky, and its test performance would regress toward the mean. Picking the representative seed gives a deployed artefact whose reported performance you can actually expect.

In this run it selected `transformer_focal`, which comes third on test cost. That looks like a worse choice than `gru_focal` — and it *is*, on test. But we don't get to know that at selection time, and the whole point of the discipline is not to pretend otherwise.

```
# src/experiment.py (staleness guard, abridged)
        if len(entry.test_scores) != len(rows_test):
            raise RuntimeError(
                f"cached scores for {entry.name} have
                  {len(entry.test_scores)} test rows but the "
                f"current split has {len(rows_test)}. The data or split
                  changed; delete "
                f"{results_dir / 'scores'} and rerun."
            )
```

The score cache is what makes the analysis re-runnable in 90 seconds. It's also a correctness hazard: change the simulator and the cached scores refer to different rows. This checks length and crashes with the remedy in the message. It would *not* catch a change that preserves row count — regenerating with a different seed, say — which is a real gap. A content hash of the feature frame would close it.

`src/report.py` renders the README's tables from `results/`. It exists so the write-up can't quote a number the artefacts don't contain. It's the least interesting file here and the one I'd most defend to someone who thinks it's over-engineering: hand-transcribed numbers rot silently, and this project's entire claim is that its numbers are trustworthy.

Part 16: packaging, CI, Docker

`pyproject.toml` configures ruff (with docstring checks on, `D` rules) and mypy with `disallow_untyped_defs`. Both are strict enough to be annoying, which is the point — mypy caught two real pandas indexing bugs in this codebase.

`.github/workflows/ci.yml` runs the leakage tests **on their own, first**:

```
      - name: Temporal leakage tests
        run: pytest tests/test_leakage.py -v
```

Separately and verbosely, so a leak is unmistakable in the log rather than one `F` among a hundred dots. It's the suite that catches the bug which silently produces incredible metrics, and it gets its own line in the CI output.

The Dockerfile installs the CPU-only torch wheel from PyTorch's own index. The default PyPI wheel drags in the CUDA runtime — several gigabytes — onto a machine with no GPU. `requirements-serve.txt` deliberately omits LightGBM, SHAP and matplotlib: the scoring container shouldn't carry the training stack, and a smaller image is a smaller attack surface.

Being honest: the image has never been built. No Docker daemon was available in the environment this was developed in, and the sandbox blocks `download.pytorch.org`. `docker compose config` validates the compose file, which is syntax only. Treat the Dockerfile as unverified.

Part 17: what's messy, and what I'd do differently

Said here rather than buried, because you asked.

The risk encodings are the weak point. They're the only thing breaking exact online/offline parity, they're the only label-consuming feature, and they carry the most machinery — temporal blocks, expanding priors, two code paths for train and non-train rows — for four features that LightGBM's native categorical handling largely duplicates. If I rebuilt this I'd try dropping target encoding entirely, keeping frequency encoding, and letting the embeddings learn merchant risk. Parity would become unconditional and the feature builder would lose a third of its complexity.

The embargo doesn't cover the longest window. It's one day; our longest velocity feature is 7 days. A transaction two days after the training cutoff can still have window entries from inside training. Since features are causal this isn't *leakage* — the model never trains on those rows — but it does mean the val/test boundary isn't as clean as "one day" implies. The embargo should be 7 days to match the longest window, and it isn't.

`experiment.py` **is a 600-line function.** `run()` does data, training, evaluation, calibration, policy, monitoring, explanation and deployment. It's readable top-to-bottom, which is why it survived, but it has eight responsibilities and no way to run stage six without stages one through five. Splitting it into a small pipeline of stage functions with explicit inputs would make partial re-runs possible.

Two of the four `entity()` **return values are discarded three times.** `is_new_mcc, since_mcc, _, _ = entity(mcc_key)` computes the window codes and seen-count for MCC, country and device type, and throws them away. It's wasted work — small, but it's the kind of thing that reads as sloppiness.

Test coverage is uneven. The leakage and serving paths are covered thoroughly. `explain.py` has essentially no direct tests — it's exercised only through the experiment run. `simulate.py`'s fraud injectors are tested for statistical properties but not for the specific mechanics of any one injector. `report.py` is untested entirely.

Config knobs I removed while writing this, because they were documented as if they did something and didn't: `n_devices`, `min_history`, `num_workers`, `ProjectConfig`/`DEFAULT_CONFIG`, `PROCESSED_DIR`, and the duplicated `CODED_CATEGORICALS`/`RAW_CATEGORICALS` constants in `schema.py` (which shadowed the live list in `features.py` and would have drifted). `true_positive_recovery` was documented in the cost model and never referenced — that one is now implemented rather than removed, since

the knob is genuinely meaningful.

The `min(ratio, 1/ratio)` symmetry in the monitor is probably wrong for production. A halving of the alert rate and a doubling are treated as equally severe. In practice a collapse means you're missing fraud and a spike means your queue is overwhelmed; those need different runbooks and probably different thresholds.

Rebuild checklist

Create files in this order. Each line is one file, and each depends only on things above it.

1. `pyproject.toml` — ruff + mypy strict, pytest config. Do this first so the tooling nags from line one.
2. `.gitignore` — exclude `data/`, `artifacts/`, `results/`, `.venv/` before you accidentally commit a 500 MB checkpoint.
3. `src/data/schema.py` — column names, `FraudType`, `PAD_INDEX`/`UNK_INDEX`. Zero dependencies.
4. `src/config.py` — every threshold and magic number, as frozen dataclasses.
5. `src/simulate.py` — universe, legitimate traffic, three fraud mechanisms, confounders, adversarial drift. Check your baseline can't hit PR-AUC 1.0 before moving on.
6. `tests/conftest.py` — a small session-scoped synthetic dataset; everything downstream tests against it.
7. `tests/test_simulate.py` — assert fraud overlaps legitimate behaviour. This is what stops you building a benchmark that measures nothing.
8. `src/data/splits.py` — temporal split with embargo. Never `train_test_split`.

9 `src/data/features.py` — `customer_causal_features` over numpy arrays first, then `FeatureBuilder` around it. Bound the lookback from the start; retrofitting it is painful.

10 `tests/test_leakage.py` — write this *before* the models. Include the has-teeth test.

11 `src/data/sequences.py` — `build_windows`, then `gather`, then the batcher. Left-pad.

12 `tests/test_features_and_sequences.py` — hand-checked velocity counts on a five-row fixture. Worth the twenty minutes.

13 `src/evaluation.py` — PR-AUC, precision at capacity by rank, per-mechanism recall, accuracy next to its refutation.

14 `src/artifacts.py` — `ModelScores`. The boundary between training and analysis; define it before you need it.

15 `src/baselines.py` — LightGBM with native categoricals and PR-AUC early stopping, plus the four-rule floor.

16 `tests/test_evaluation.py` — especially the tied-scores alert-volume test.

17 `src/models/base.py` — `ModelConfig`, `InputEncoder`, `SequenceClassifier` ABC, `roll_to_front`, `gather_at`.

18 `src/models/rnn.py` — roll, run, gather at `length - 1`.

19 `src/models/tcn.py` — left-pad-and-slice causal convolutions, re-mask after every block.

20 `src/models/transformer.py` — index-comparison masks, diagonal always open, fused and explicit attention paths.

21 `src/models/__init__.py` — registry, per-architecture defaults, `make_config`.

22 `tests/test_models.py` — padding-invariance test above all others.

23 `src/train.py` — focal loss, weighted sampler, PR-AUC early stopping with `deepcopy`, multi-seed.

24 `tests/test_train.py` — focal-loss properties, weighted sampling actually resamples, same seed reproduces.

25 `src/calibration.py` — quantile-binned ECE, alert-region calibration, temperature and isotonic.

26 `src/policy.py` — cost model, quantile threshold sweep, choose on validation, aggregate per seed.

27 `tests/test_calibration_and_policy.py` — assert test cost never beats the oracle; that's your selection-leakage detector.

28 `src/monitoring.py` — PSI with fixed reference edges, alert-rate ratio, per-window per-mechanism recall.

29 `tests/test_monitoring.py` — train before the drift, monitor across it, assert the alarm fires.

30 `src/explain.py` — attention, gradient×input, SHAP, worked example. Ship the caveat with the output.

31 `src/api/schemas.py` — pydantic request/response. No `is_fraud` field anywhere, including on history.

32 `src/api/service.py` — `required_history` arithmetic, ONNX export, calibrate, decide.

33 `src/api/audit.py` — append-only JSONL, flush per record, content hash.

34 `src/api/app.py` — lifespan load, `/score`, `/health`, `/model-info`, `/metrics`.

35 `tests/test_serving.py` — online/offline parity is the test that justifies the whole design of `features.py`.

36 `src/api/benchmark.py` — time the model *and* the feature build separately.

37 `src/experiment.py` — one command, per-seed everything, select on validation.

38 `src/report.py` — render the write-up's tables from the artefacts.

39 `tests/test_metric_regression.py` — floors at half of observed, plus a ceiling. PR-AUC above 0.995 means leakage, not skill.

40 `Dockerfile`, `requirements-serve.txt`, `docker-compose.yml` — CPU torch, serving deps only.

41 `.github/workflows/ci.yml` — leakage suite first and alone.

42 `README.md`, `DECISIONS.md`, `MODEL_CARD.md` — last, from the artefacts, never from memory.

The two places you will lose a day if you get them wrong: the padding convention (step 11, and it silently poisons step 17 onward), and the `required_history` arithmetic (step 32, which you won't discover until step 35). Write the tests at steps 10 and 35 before the code they cover.